Meeting 10 — Failure Handling & Fault Tolerance Architecture

Project: Digital Harmonium Workstation

Version: Final Design Meeting

Status: Finalized

10.1 Purpose

A commercial musical instrument is judged not only by how well it performs under ideal conditions, but also by how gracefully it behaves when things go wrong.

The objective of this meeting is to define the workstation's fault-tolerance philosophy.

Rather than asking:

> "How should the system work?"

this meeting asks:

> "How should the system behave when something fails?"

Every predictable failure should have:

- A detection mechanism
- A recovery strategy
- A user-visible response (if necessary)
- A guarantee that the system remains stable whenever possible

The workstation must always attempt graceful degradation rather than unexpected failure.

10.2 Fundamental Engineering Philosophy

The following design philosophy shall govern the entire software architecture.

Principle 1 — Music Comes First

The workstation is a musical instrument.

Everything else exists to support music.

If a conflict occurs between audio generation and any secondary subsystem, audio always receives priority.

Examples:

- Skip LCD updates before interrupting audio.
- Pause SD card writes before interrupting audio.
- Delay background maintenance before interrupting audio.
- Stop recording before interrupting audio.
- Disable nonessential features before interrupting audio.

Sound generation must always be protected.

Principle 2 — Fail Gracefully

The system should never crash because of a recoverable fault.

Whenever possible:

- Continue operating.
- Disable only the affected feature.
- Preserve user data.
- Inform the user clearly.

Principle 3 — Never Lose Existing Work

Whenever practical:

- Existing recordings remain intact.
- Installed instruments remain usable.
- User settings remain recoverable.
- Corrupted data is isolated instead of spreading.

10.3 Failure Classification

Failures are divided into twelve engineering categories.

Category 1 — Storage Failures

Examples:

- SD card missing
- SD card removed
- File not found
- Corrupted filesystem
- Read timeout
- Write timeout
- Card becomes read-only

Recovery:

- Current instrument continues because it resides in Flash.
- Recording stops safely.
- Instrument loading becomes unavailable.
- Playback becomes unavailable.
- Display informative warning.

System continues operating.

Category 2 — Instrument Failures

Examples:

- Invalid VHL package
- CRC mismatch
- Unsupported version
- Missing metadata
- Missing samples
- Corrupted instrument file

Recovery:

- Cancel installation.
- Preserve previously installed instrument.
- Notify user.
- Never install partial instruments.

Category 3 — Memory Failures

Memory architecture is intentionally designed to eliminate runtime allocation.

Rule

All major runtime objects are allocated during boot.

During performance:

- No dynamic allocation
- No heap growth
- No runtime fragmentation

This removes an entire class of embedded failures.

Voice Pool Exhaustion

Situation:

More notes requested than available voices.

Recovery:

Voice stealing.

Priority:

1. Released voices
2. Oldest active voice

The workstation never allocates additional memory.

Queue Overflow

Situation:

Producer creates events faster than consumer processes them.

Recovery:

Discard redundant scan events.

Critical note events are preserved.

Recorder Buffer Full

Situation:

Storage cannot keep pace with recording.

Recovery:

80%

Increase Storage Task priority.

95%

Display warning.

100%

Stop recording.

Music continues.

Heap Fragmentation

Prevented completely.

No runtime allocation is performed during musical operation.

Stack Overflow

Each FreeRTOS task uses fixed stack memory.

Overflow detection remains enabled.

If detected:

- Log event.
- Restart subsystem when possible.
- Otherwise perform controlled reboot.

10.4 Audio Failures

Audio is the highest priority subsystem.

DMA Underrun

Situation:

DMA requests audio before CPU prepares next buffer.

Recovery:

Double-buffer architecture prevents starvation.

Worst case:

Repeat previous buffer once.

Small click is preferable to silence or crash.

Audio Generation Overrun

Situation:

CPU cannot complete synthesis before next DMA deadline.

Recovery:

Finish current safe buffer.

Resume normal generation immediately.

Audio timing always preserved.

Mixer Overflow

Situation:

Summed voices exceed digital range.

Recovery:

Limiter prevents clipping.

Output remains within valid range.

Excessive Polyphony

Situation:

More notes requested than supported.

Recovery:

Voice stealing.

Never exceed fixed voice pool.

Corrupted Sample

Situation:

Single sample becomes unreadable.

Recovery:

Mute affected voice only.

Entire instrument remains operational.

I^2S Communication Fault

Recovery:

Reinitialize I^2S peripheral.

System reboot only if recovery fails repeatedly.

Amplifier Failure

Software continues normally.

No software recovery required.

10.5 Timing Failures

Real-time deadlines are continuously protected.

Display Delay

Display updates are skipped.

Audio continues uninterrupted.

Background Task Delay

Background maintenance pauses immediately.

Higher-priority tasks execute first.

Instrument Loader Delay

Loader pauses whenever audio requires CPU time.

Installation resumes afterward.

Recorder Delay

Recording yields immediately to audio.

Buffered data written later.

Missed Task Deadline

Repeated misses increment diagnostic counters.

Persistent failures generate warnings.

10.6 User Input Failures

Examples:

- Button bounce
- Multiple simultaneous presses
- Long presses
- Rapid repeated presses
- Slider noise
- Matrix ghosting

Recovery:

- Software debouncing
- Priority rules
- Invalid combinations ignored
- Stable input filtering

10.7 Recording Failures

Examples:

- SD full
- Recording interrupted
- Power loss
- Corrupted playback file

Recovery:

- Flush buffers whenever possible.
- Preserve completed recordings.
- Stop recording safely.
- Never overwrite previous recordings unintentionally.

10.8 Boot Failures

Examples:

- Missing settings
- Corrupted settings
- Missing instrument
- Interrupted startup

Recovery:

Load factory defaults.

Always attempt successful boot.

A missing instrument must never prevent startup.

10.9 Hardware Peripheral Failures

Examples:

- LCD disconnected
- LEDs malfunction
- Speaker unplugged
- Headphones disconnected

Recovery:

Continue normal operation whenever possible.

Peripheral failure should never stop musical performance.

10.10 Configuration Failures

Examples:

- Invalid transpose
- Invalid tempo
- Unsupported preset

Recovery:

Clamp values.

Load defaults.

Continue operation.

10.11 Background Service Failures

Examples:

- Delayed cleanup
- Memory statistics unavailable
- Diagnostic update skipped

Recovery:

Retry during next maintenance cycle.

Background services never interrupt audio.

10.12 Unknown Failures

Not every failure can be predicted.

Every subsystem therefore follows the same recovery pattern.

```
Detect

↓

Report

↓

Recover

↓

Continue
```

Unexpected failures should never immediately terminate the entire workstation unless safe operation is impossible.

10.13 Failure Severity Levels

All detected faults are classified.

Level	Meaning	Typical Action
Level 0	Automatic recovery	User never notices
Level 1	Minor warning	Continue normally
Level 2	Feature disabled	Music continues
Level 3	User intervention required	Await correction
Level 4	Critical system fault	Controlled restart or shutdown

10.14 Priority Preservation

Whenever CPU resources become limited, tasks are sacrificed in the following order:

```
DMA

↓

Audio Processing

↓

Keyboard

↓

Voice Manager

↓

Recorder

↓

Display

↓
```

```
Instrument Loader

↓

Background Maintenance
```

Higher-priority systems may interrupt lower-priority systems at any time.

Lower-priority systems must never delay audio generation.

10.15 Engineering Rules

The following rules become permanent design requirements.

- Runtime memory allocation is prohibited during musical performance.
- Audio generation always has highest execution priority.
- Installed instruments remain usable even if the SD card is removed.
- Voice stealing replaces dynamic allocation.
- Every subsystem attempts graceful recovery before failure.
- Existing recordings and settings are preserved whenever possible.
- Fault messages should be understandable by musicians rather than engineers.
- Partial failures should affect only the relevant subsystem.
- Background services yield immediately whenever real-time tasks require CPU time.
- Every recoverable error must have a deterministic recovery procedure.

10.16 Final Design Principle

The Digital Harmonium Workstation shall always sacrifice convenience before sacrificing sound.

Display updates may wait.

Storage writes may wait.

Diagnostics may wait.

Background maintenance may wait.

Music must not.

This principle governs every architectural decision made throughout the project.

10.17 Summary

Meeting 10 completes the architectural design of the Digital Harmonium Workstation.

Rather than treating failures as exceptional events, the workstation is designed with failure awareness from the beginning.

Every major subsystem has predictable recovery behavior.

Every recoverable fault has a defined response.

The result is a workstation that remains stable, responsive, and musically useful even under adverse conditions.

This concludes the complete high-level engineering design phase of the Digital Harmonium Workstation.

End of Meeting 10 — Failure Handling & Fault Tolerance Architecture